Application de gestion de profils utilisateurs

Base de données : **MongoDB** (orientée document)

API : **Fast API (Python)** + PyMongo | Frontend : **HTML/CSS/JS** | Infrastructure : **Docker Compose**

Kylian Guillermier · 23 April 2026

Projet NoSQL

1. Modélisation de la base de données

Pourquoi MongoDB ?

MongoDB est une base de données NoSQL orientée document. Contrairement aux bases relationnelles traditionnelles (MySQL, PostgreSQL), elle ne requiert pas de schéma fixe : chaque document JSON peut avoir sa propre structure, ce qui le rend particulièrement adapté à ce projet.

- **Profils flexibles** : chaque utilisateur peut posséder un nombre variable d'adresses sans nécessiter de table de jointure supplémentaire.
- **Imbrication native** : les sous-documents (*addresses*, *preferences*) sont stockés directement dans le document parent, ce qui simplifie les lectures.
- **Schéma évolutif** : de nouveaux champs peuvent être ajoutés à tout moment sans migration coûteuse.
- **JSON natif** : l'API Fast API renvoie du JSON ; MongoDB stocke en BSON (Binary JSON), éliminant toute couche de conversion.

Schéma du document — Collection *users*

```
{
  "_id"        :
  ObjectId("664a1f2e3b7c8d9e0f1a2b3c"), "name" :
  "Alice Dupont",
  "email"      :
  "alice@example.com", "addresses" :
  [
    { "street": "12 rue de la Paix", "city": "Paris",    "country": "France"      },
    { "street": "5 Baker Street",     "city": "Londres", "country": "Royaume-Uni" }
  ],
  "preferences" : {
    "theme"         : "dark",
    "notifications" : true
  },
  "created_at" : "2026-04-23T10:00:00+00:00"
}
```

Description des champs

Champ	Type	Description
_id	ObjectId	Identifiant unique généré automatiquement par MongoDB
name	String	Nom complet de l'utilisateur
email	String	Adresse e-mail de l'utilisateur (non unique contraint ici)
addresses	Array<Object>	Tableau d'objets contenant street, city et country
preferences	Object	Objet imbriqué : thème (dark/light) et notifications (bool)
created_at	String (ISO)	Horodatage de création au format ISO 8601 UTC

2. Architecture de l'application

L'application repose sur trois services orchestrés par **Docker Compose**. Chaque service tourne dans un conteneur isolé et communique via un réseau Docker interne.

Service	Image / Technologie	Port exposé	Rôle
api	Python 3.12 + Fast API	8000	Expose les routes REST CRUD et la requête avancée
frontend	Nginx + HTML/CSS/JS	3000	Interface utilisateur statique, appels fetch() vers l'API
mongodb	mongo:7.0	27017	Stockage persistant des documents utilisateurs

Flux de communication

Le navigateur accède au frontend Nginx sur le port **3000**. Les actions utilisateur (création, modification, suppression) déclenchent des requêtes **fetch()** vers l'API Fast API sur le port **8000**. Fast API interroge ensuite MongoDB sur le port **27017** via le driver PyMongo. La réponse JSON remonte jusqu'au navigateur.



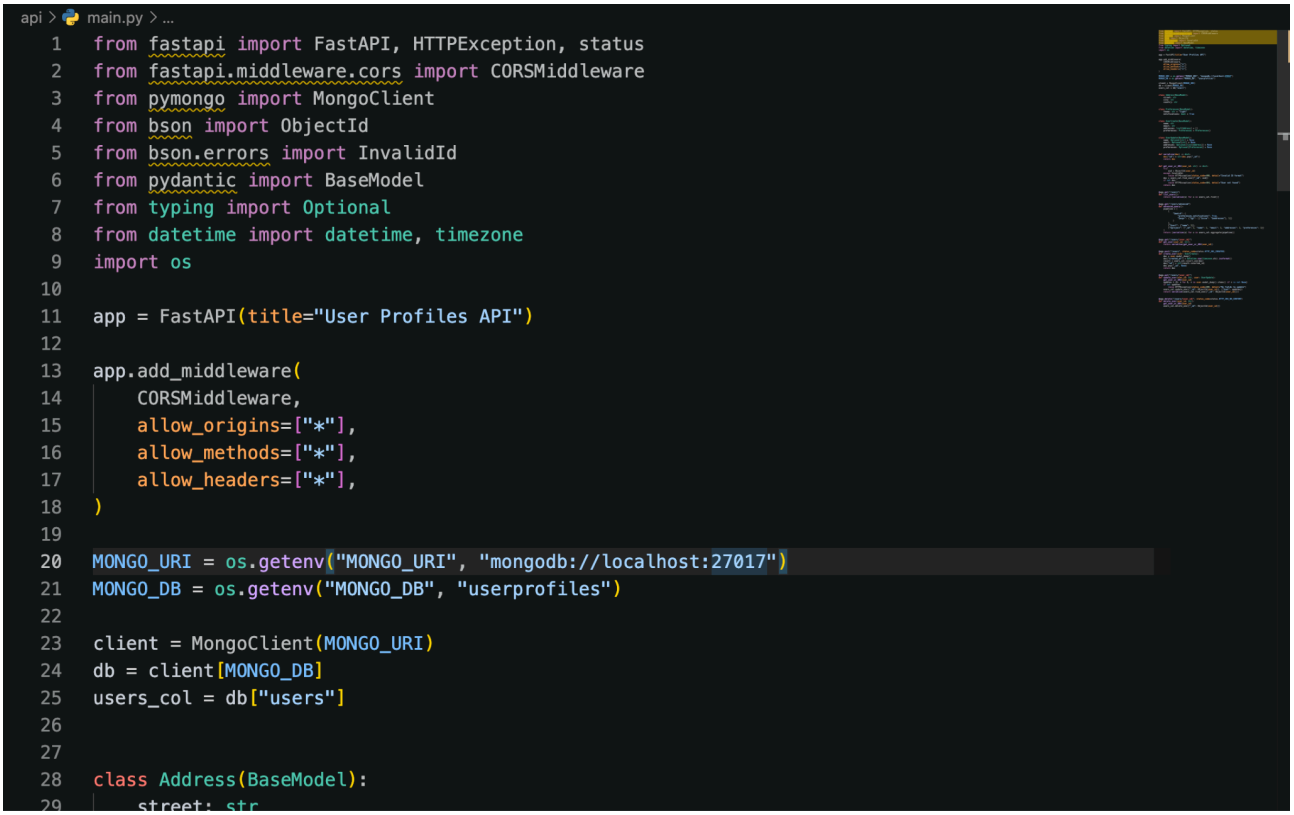
Variables d'environnement

```
# docker-compose.yml (extrait)
environment:
  MONGO_URI: mongodb://mongodb:27017
  MONGO_DB: userprofiles
```

3. Code Python — api/main.py

Imports et connexion MongoDB (lignes 1–17)

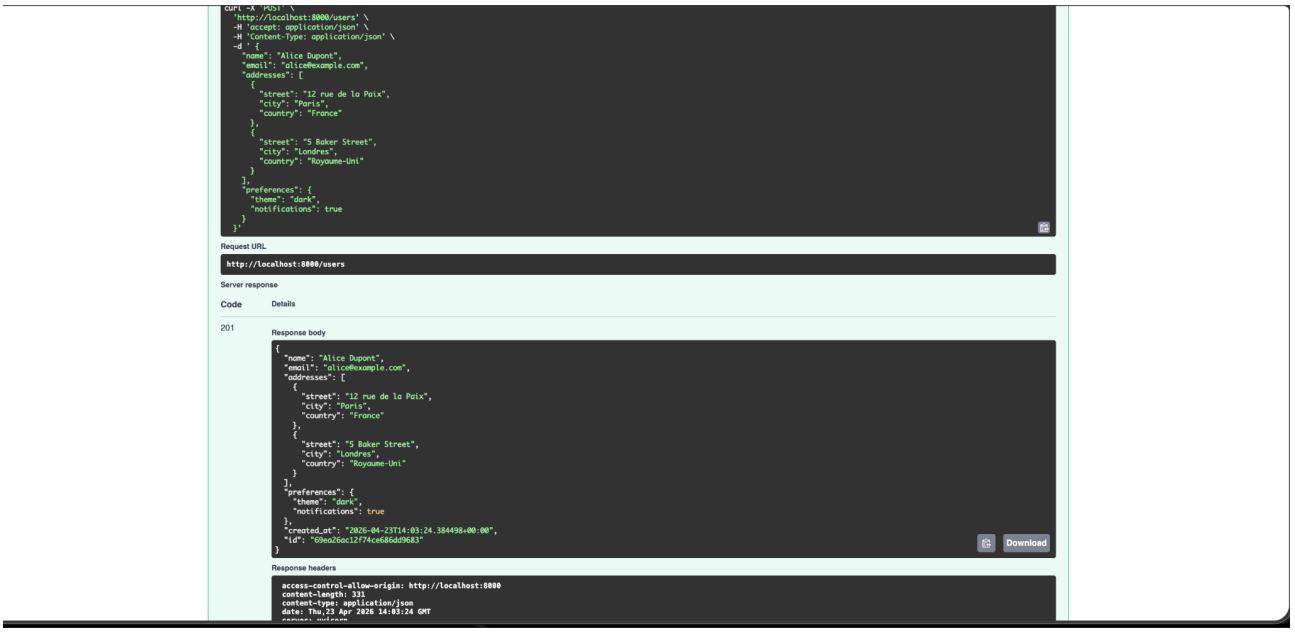
Le fichier commence par l'import des bibliothèques nécessaires : **Fast API** pour le serveur HTTP, **PyMongo** pour interagir avec MongoDB, et **bson** pour la conversion des ObjectId. La connexion est initialisée via la variable d'environnement **MONGO_URI** injectée par Docker Compose.



Routes CRUD

CREATE — POST /users

```
@app.post("/users", status_code=201)
async def create_user(user:
UserCreate):
    data = user.dict()
    data["created_at"] =
datetime.now(timezone.utc).isoformat()
    result = users_col.insert_one(data)
    data["id"] =
str(result.inserted_id)
    data.pop("_id", None)
    return data
```



READ — GET /users

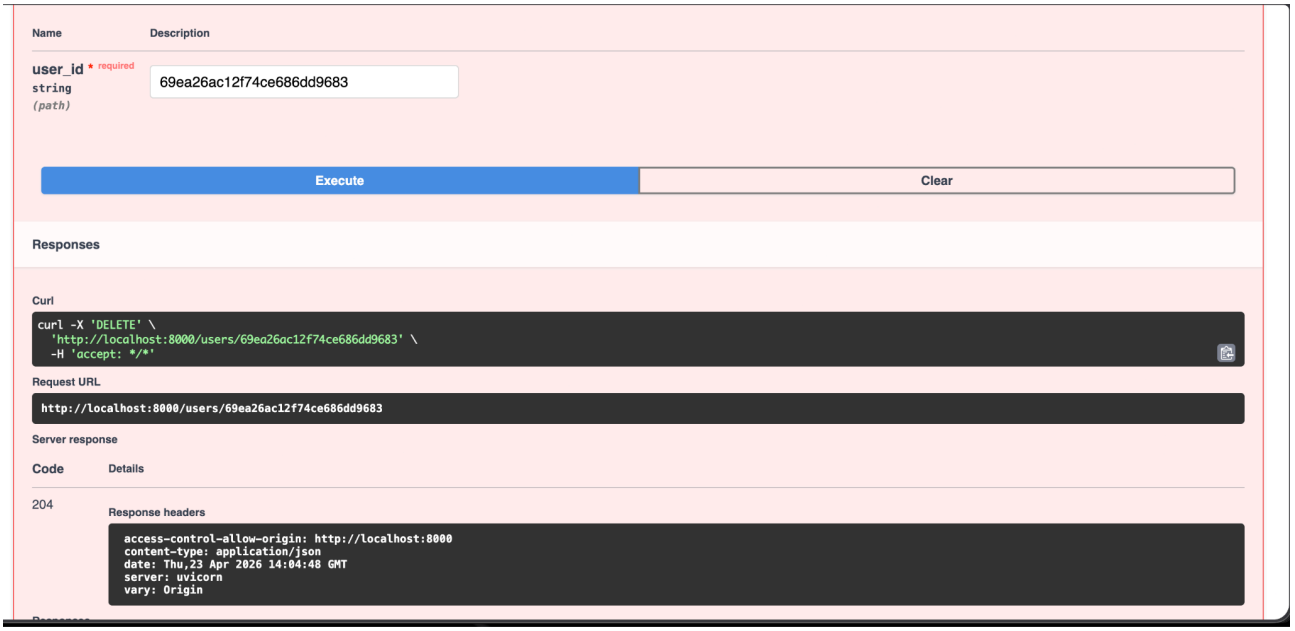
```
@app.get("/users")
async def get_users():
    result = list(users_col.find())
    return [serialize(u) for u in result]
```

UPDATE — PUT /users/

```
@app.put("/users/{user_id}")
async def update_user(user_id: str, user: UserUpdate):
    users_col.update_one(
        {"_id": ObjectId(user_id)},
        {"$set": user.dict(exclude_unset=True)}
    )
    updated = users_col.find_one({"_id": ObjectId(user_id)})
    return serialize(updated)
```

DELETE — DELETE /users/

```
@app.delete("/users/{user_id}")
async def delete_user(user_id: str):
    users_col.delete_one({"_id": ObjectId(user_id)})
    return {"message": "Utilisateur supprimé avec succès"}
```



Requête avancée — Pipeline d'agrégation

La route **GET /users/advanced** filtre les utilisateurs qui ont les notifications activées *et* possèdent plus d'une adresse. Elle utilise un pipeline MongoDB composé de deux étapes :

- **\$match** : sélectionne les documents où *preferences.notifications = true* et où la taille du tableau *addresses* est supérieure à 1.
- **\$sort** : trie les résultats par ordre alphabétique sur le champ *name*.

```
@app.route("/users/advanced", methods=["GET"])
def advanced_query():
    pipeline = [
        {
            "$match": {
                "preferences.notifications": True,
                "$expr": {"$gt": [{"$size": "$addresses"}, 1]}
            }
        },
        {"$sort": {"name": 1}}
    ]
    result = list(users.aggregate(pipeline))
    return jsonify([serialize(u) for u in result])
```

4. Démonstration des opérations CRUD

Les captures ci-dessous illustrent chaque opération exécutée via **curl** depuis un terminal. L'API tourne sur *http://localhost:8000* et MongoDB sur le port *27017*.

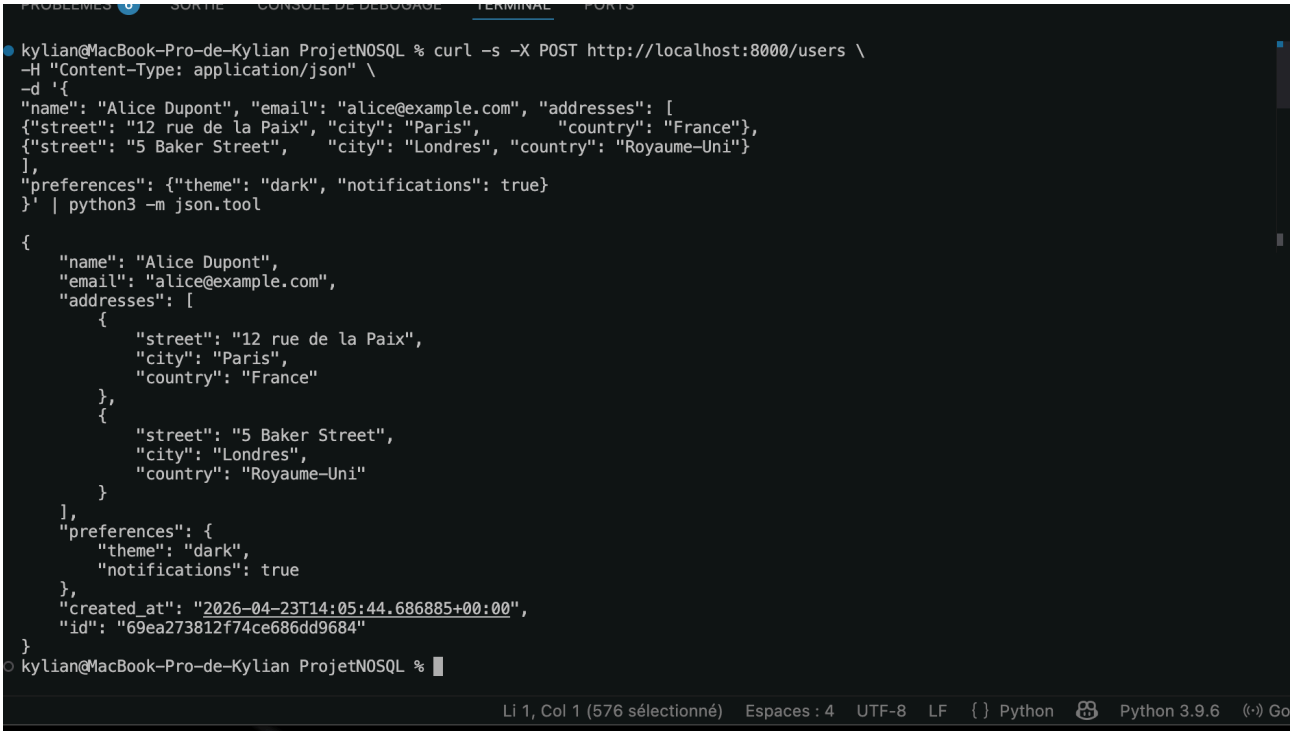
CREATE — POST /users

Création d'un utilisateur avec deux adresses et des préférences :

```
curl -s -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Alice Dupont",
    "email": "alice@example.com",
    "addresses": [
      {"street": "12 rue de la Paix", "city": "Paris",   "country": "France"},
      {"street": "5 Baker Street",    "city": "Londres", "country": "Royaume-Uni"}
    ],
    "preferences": {"theme": "dark", "notifications": true}
  }' | python3 -m json.tool
```

Réponse attendue (HTTP 201) :

```
{
  "id"         : "664a1f2e3b7c8d9e0f1a2b3c",
  "name"       : "Alice Dupont",
  "email"      :
  "alice@example.com", "addresses" :
  [ ... ],
  "preferences": { "theme": "dark", "notifications": true },
  "created_at" : "2026-04-23T10:00:00+00:00"
```



READ — GET /users

Récupération de la liste complète des utilisateurs :

```
curl -s http://localhost:8000/users | python3 -m json.tool
```

Réponse attendue : tableau JSON contenant tous les documents de la collection.

→ kylian@MacBook-Pro-de-Kylian ProjetNOSQL % curl -s http://localhost:8000/users | python3 -m json.tool

[
{
"name": "GOD MOI",
"email": "tg@gmail.com",
"addresses": [
{
"street": "HJOFBJBFHUOIFHIOFHIOFHIOHIOHIO",
"city": "",
"country": ""
}
],
"preferences": {
"theme": "dark",
"notifications": true
},
"created_at": "2026-04-23T12:56:24.994513+00:00",
"id": "69ea16f8861c45169efd4c46"
},
{
"name": "string",
"email": "string",
"addresses": [],
"preferences": {
"theme": "light",
"notifications": true
},
"created_at": "2026-04-23T14:00:54.952687+00:00",
"id": "69ea261612f74ce686dd9682"
},
{
"name": "Alice Dupont",
"email": "alice@example.com",
"addresses": [
{
"street": "12 rue de la Paix",
"city": "Paris",
"country": "France"

},

{

"street": "5 Baker Street",

"city": "Londres",

"country": "Royaume-Uni"

}

],

"preferences": {

"theme": "dark",

"notifications": true

},

"created_at": "2026-04-23T14:05:44.686885+00:00",

"id": "69ea273812f74ce686dd9684"

}

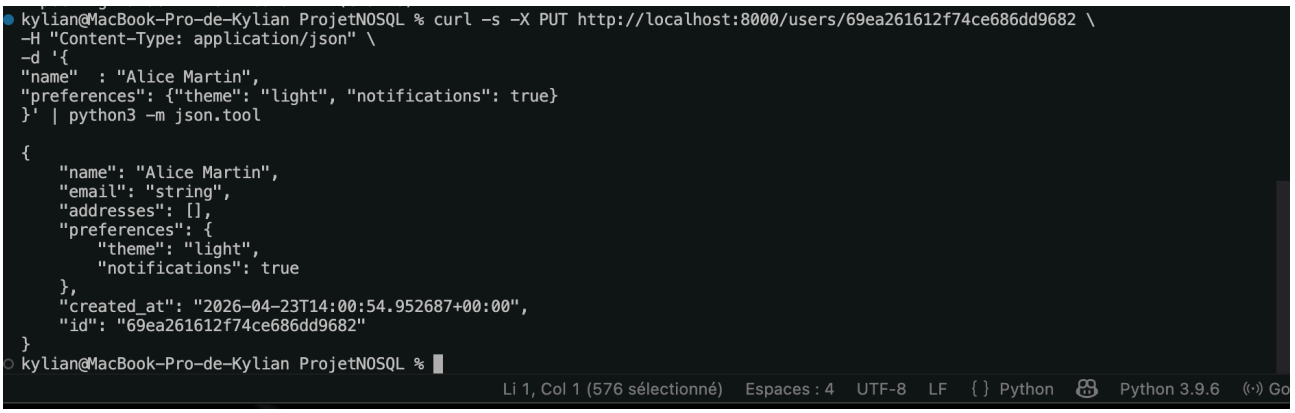
]

kylian@MacBook-Pro-de-Kylian ProjetNOSQL %

UPDATE — PUT /users/

Modification du nom et du thème de l'utilisateur créé précédemment (remplacer <*ID*> par l'id retourné lors du CREATE) :

```
curl -s -X PUT http://localhost:8000/users/<ID> \
  -H "Content-Type: application/json" \
  -d '{
    "name"        : "Alice Martin",
    "preferences": {"theme": "light", "notifications": true}
  }' | python3 -m json.tool
```



Réponse attendue : document complet avec *name* mis à jour à **Alice Martin**.

DELETE — DELETE /users/

```
curl -s -X DELETE http://localhost:8000/users/<ID> | python3 -m json.tool
```

Réponse attendue :

```
{ "message": "Utilisateur supprime" }
```



Requête avancée — GET /users/advanced

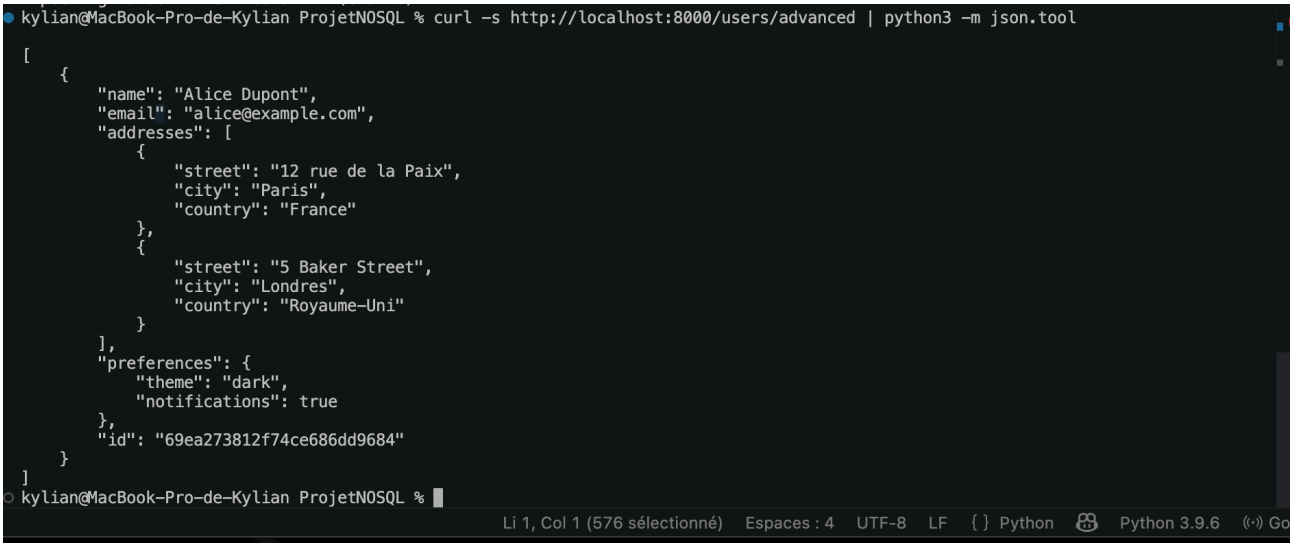
Avant d'exécuter cette requête, créer au moins deux utilisateurs ayant *notifications* = *true* et deux adresses. La route filtre et trie ces profils :

```
curl -s http://localhost:8000/users/advanced | python3 -m json.tool
```

Pipeline MongoDB exécuté :

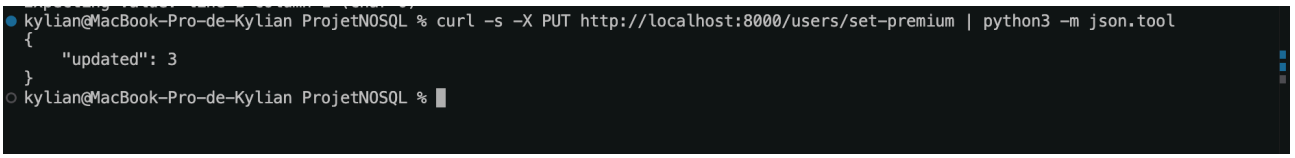
```
Etape 1 — $match :
  - preferences.notifications = true
  - taille de addresses > 1 ($expr + $size)

Etape 2 — $sort :
  - tri alphabetique croissant sur name
```

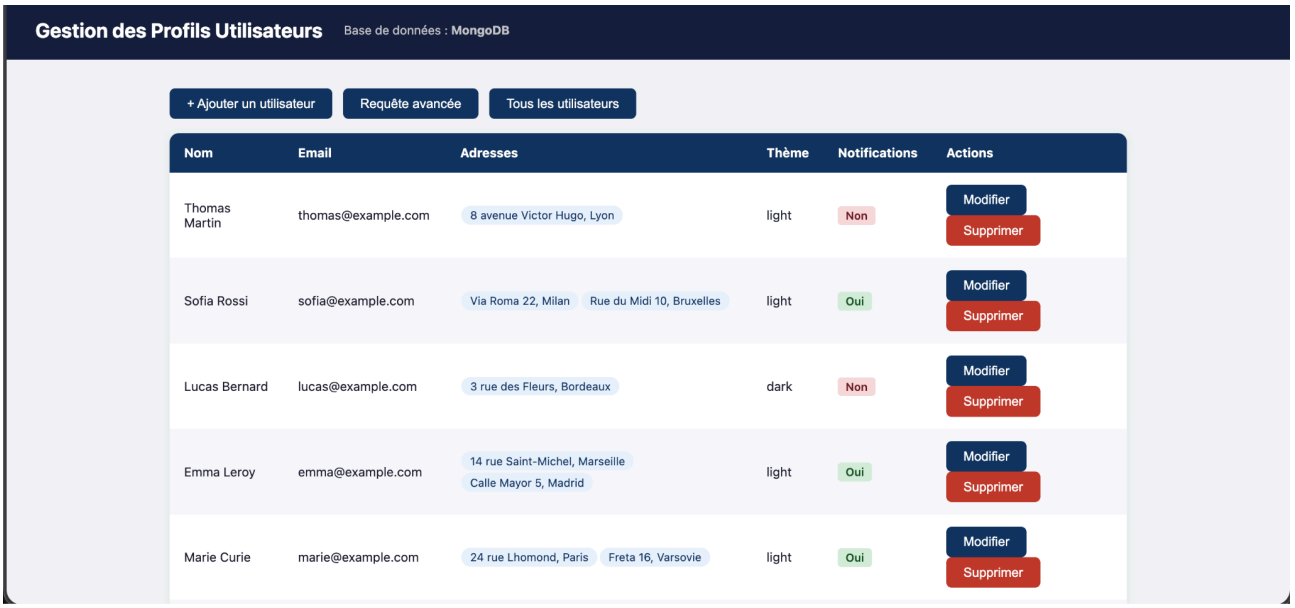


Mise à jour conditionnelle — Ajout du statut "premium"

Une requête avancée de mise à jour conditionnelle a également été implémentée. Elle cible uniquement les utilisateurs répondant à un critère précis : avoir activé les notifications dans leurs préférences. Pour ces utilisateurs, un champ premium est automatiquement ajouté et défini à true, sans affecter les autres profils. Cette opération utilise l'opérateur MongoDB $set combiné à un filtre sur preferences.notifications, ce qui illustre la flexibilité du modèle document : il est possible d'enrichir dynamiquement un sous-ensemble de documents sans modifier la structure globale de la collection. Le résultat est visible sur la capture ci-dessous, où le champ premium apparaît bien dans le document mis à jour.



Frontend :



Lien Github : https://github.com/Kyliantvv/projet-mangodb

Conclusion

Ce qui fonctionne

- Les quatre opérations CRUD sont opérationnelles et testées via curl.
- La requête avancée avec pipeline d'agrégation MongoDB filtre et trie correctement les profils.
- L'architecture Docker Compose isole proprement les trois services (API, frontend, base de données).
- La sérialisation des ObjectId en string permet une intégration JSON transparente.
- CORS est configuré, ce qui autorise les appels depuis le frontend Nginx.